bit_analysis_2m

July 10, 2025

1 Análisis de eficiencia de bits 2 metros de distancia

Este notebook analiza qué tan parecidos son los bits recibidos `bits_rx` con los bits transmitidos `bits_tx` y presenta gráficas, métricas de rendimiento y las imágenes que no cumplen con el criterio de coincidencia.

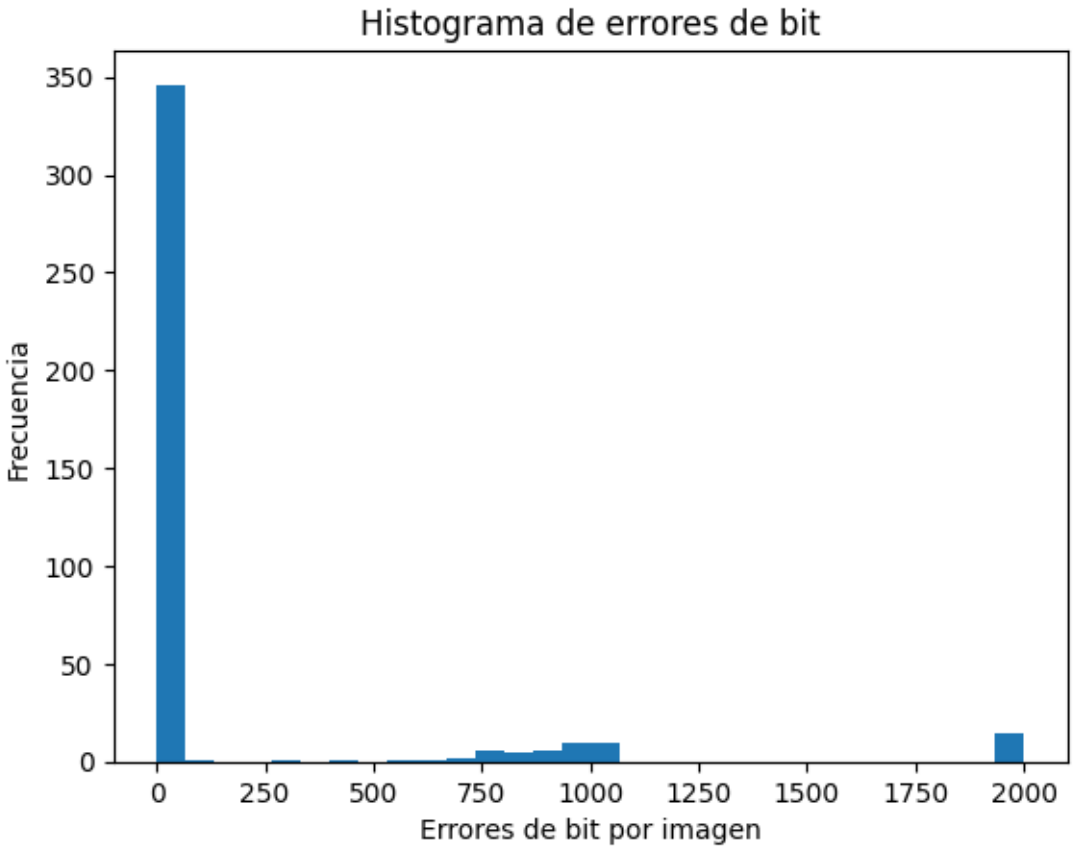
[16]:
```
import pandas as pd
from pathlib import Path
from IPython.display import display, Image
import matplotlib.pyplot as plt

path = "./results/2m/"
nombre_csv = "resultados_comparacion_2m.csv"
df = pd.read_csv(path + nombre_csv)

# Vista rápida del DataFrame
display(df.head())
print(f"Filas totales: {len(df)}")
```

```
   num_auto                       bw_plate_tx  \
0         1  dataset_yolo/autos_bw/auto001.png
1         2  dataset_yolo/autos_bw/auto002.png
2         2  dataset_yolo/autos_bw/auto002.png
3         2  dataset_yolo/autos_bw/auto002.png
4         2  dataset_yolo/autos_bw/auto002.png

                        bw_plate_rx ocr_txt_tx  bits_equal  bit_errors  \
0  data_1m/autos_bw/auto001_rep01.png    KOZ.808       False           5
1  data_1m/autos_bw/auto002_rep01.png    MAU.096       False           3
2  data_1m/autos_bw/auto002_rep02.png    MAU.096       False           3
3  data_1m/autos_bw/auto002_rep03.png    MAU.096       False           5
4  data_1m/autos_bw/auto002_rep04.png    MAU.096       False          15

             bits_tx            bits_rx
0  [0 0 1 … 0 0 0]  [0 0 1 … 0 0 0]
1  [0 0 0 … 0 0 0]  [0 0 0 … 0 0 0]
2  [0 0 0 … 0 0 0]  [0 0 0 … 0 0 0]
3  [0 0 0 … 0 0 0]  [0 0 0 … 0 0 0]
```

```
4  [0 0 0 … 0 0 0]  [0 0 0 … 0 0 0]

Filas totales: 404
```

1.1 Distribución de errores de bit por imagen

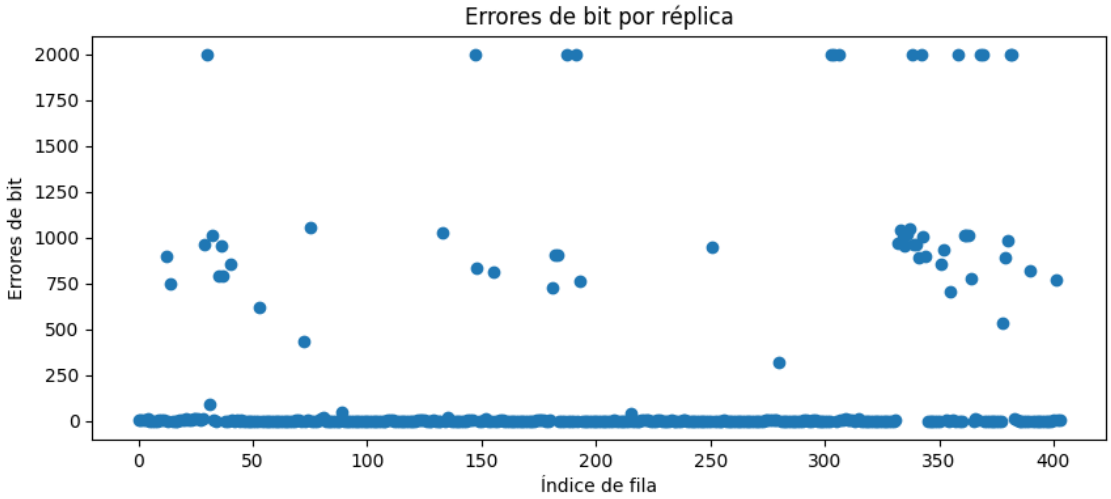
[17]:
```
plt.figure()
df['bit_errors'].plot(kind='hist', bins=30)
plt.xlabel('Errores de bit por imagen')
plt.ylabel('Frecuencia')
plt.title('Histograma de errores de bit')
plt.show()
```



1.2 Errores de bit por réplica

[18]:
```
plt.figure(figsize=(10,4))
plt.plot(df.index, df['bit_errors'], marker='o', linestyle='none')
plt.xlabel('Índice de fila')
plt.ylabel('Errores de bit')
plt.title('Errores de bit por réplica')
```

```
plt.show()
```



1.3 Imágenes que no cumplen el criterio

[19]:
```
# Eliminamos las que tienen 2000 errores, ya que es la imagen inversa
df = df[df['bit_errors'] < 2000]
```

[20]:
```
import numpy as np   # por si no estaba importado

# --- Parámetros de tolerancia ---
BITS_TOTAL = 2000
max_errors = 100  # máximo de bits erróneos permitidos
max_ber    = max_errors / BITS_TOTAL  # % BER máximo

# --- BER por fila ---
df['bits_total'] = BITS_TOTAL
df['ber'] = df['bit_errors'] / BITS_TOTAL

# --- Criterio de falla ---
fail_df = df[(df['bit_errors'] > max_errors) | (df['ber'] > max_ber)]

print(f'Imágenes que superan el umbral: {len(fail_df)}')
print(f'Total de imágenes: {len(df)}')
print(f'Porcentaje de fallos: {len(fail_df) / len(df) * 100:.2f}%')
display(
    fail_df[['num_auto', 'bw_plate_tx', 'bw_plate_rx',
             'bit_errors', 'ber']]
)
```

```
Imágenes que superan el umbral: 43
```

Total de imágenes: 390
Porcentaje de fallos: 11.03%

	num_auto	bw_plate_tx
12	128	dataset_yolo/autos_bw/auto128.png
14	64	dataset_yolo/autos_bw/auto064.png
29	129	dataset_yolo/autos_bw/auto129.png
32	97	dataset_yolo/autos_bw/auto097.png
35	64	dataset_yolo/autos_bw/auto064.png
36	132	dataset_yolo/autos_bw/auto132.png
37	64	dataset_yolo/autos_bw/auto064.png
40	66	dataset_yolo/autos_bw/auto066.png
53	29	dataset_yolo/autos_bw/auto029.png
72	30	dataset_yolo/autos_bw/auto030.png
75	178	dataset_yolo/autos_bw/auto178.png
133	89	dataset_yolo/autos_bw/auto089.png
148	2	dataset_yolo/autos_bw/auto002.png
155	36	dataset_yolo/autos_bw/auto036.png
181	80	dataset_yolo/autos_bw/auto080.png
182	93	dataset_yolo/autos_bw/auto093.png
183	93	dataset_yolo/autos_bw/auto093.png
193	64	dataset_yolo/autos_bw/auto064.png
251	114	dataset_yolo/autos_bw/auto114.png
280	126	dataset_yolo/autos_bw/auto126.png
332	129	dataset_yolo/autos_bw/auto129.png
333	59	dataset_yolo/autos_bw/auto059.png
334	104	dataset_yolo/autos_bw/auto104.png
335	29	dataset_yolo/autos_bw/auto029.png
336	63	dataset_yolo/autos_bw/auto063.png
337	165	dataset_yolo/autos_bw/auto165.png
339	72	dataset_yolo/autos_bw/auto072.png
340	30	dataset_yolo/autos_bw/auto030.png
341	20	dataset_yolo/autos_bw/auto020.png
343	78	dataset_yolo/autos_bw/auto078.png
344	15	dataset_yolo/autos_bw/auto015.png
351	32	dataset_yolo/autos_bw/auto032.png
352	32	dataset_yolo/autos_bw/auto032.png
355	159	dataset_yolo/autos_bw/auto159.png
361	96	dataset_yolo/autos_bw/auto096.png
362	96	dataset_yolo/autos_bw/auto096.png
363	96	dataset_yolo/autos_bw/auto096.png
364	64	dataset_yolo/autos_bw/auto064.png
378	169	dataset_yolo/autos_bw/auto169.png
379	2	dataset_yolo/autos_bw/auto002.png
380	130	dataset_yolo/autos_bw/auto130.png
390	176	dataset_yolo/autos_bw/auto176.png
401	26	dataset_yolo/autos_bw/auto026.png

	bw_plate_rx	bit_errors	ber
12	data_1m/autos_bw/auto128_rep01.png	901	0.4505
14	data_1m/autos_bw/auto064_rep01.png	748	0.3740
29	data_1m/autos_bw/auto129_rep01.png	962	0.4810
32	data_1m/autos_bw/auto097_rep01.png	1014	0.5070
35	data_1m/autos_bw/auto064_rep02.png	793	0.3965
36	data_1m/autos_bw/auto132_rep01.png	953	0.4765
37	data_1m/autos_bw/auto064_rep03.png	790	0.3950
40	data_1m/autos_bw/auto066_rep01.png	852	0.4260
53	data_1m/autos_bw/auto029_rep01.png	616	0.3080
72	data_1m/autos_bw/auto030_rep02.png	431	0.2155
75	data_1m/autos_bw/auto178_rep01.png	1058	0.5290
133	data_1m/autos_bw/auto089_rep01.png	1030	0.5150
148	data_1m/autos_bw/auto002_rep05.png	834	0.4170
155	data_1m/autos_bw/auto036_rep05.png	810	0.4050
181	data_1m/autos_bw/auto080_rep03.png	724	0.3620
182	data_1m/autos_bw/auto093_rep01.png	902	0.4510
183	data_1m/autos_bw/auto093_rep02.png	902	0.4510
193	data_1m/autos_bw/auto064_rep07.png	764	0.3820
251	data_1m/autos_bw/auto114_rep02.png	948	0.4740
280	data_1m/autos_bw/auto126_rep01.png	323	0.1615
332	data_1m/autos_bw/auto129_rep05.png	972	0.4860
333	data_1m/autos_bw/auto059_rep03.png	1044	0.5220
334	data_1m/autos_bw/auto104_rep05.png	997	0.4985
335	data_1m/autos_bw/auto029_rep04.png	957	0.4785
336	data_1m/autos_bw/auto063_rep05.png	1019	0.5095
337	data_1m/autos_bw/auto165_rep01.png	1045	0.5225
339	data_1m/autos_bw/auto072_rep03.png	962	0.4810
340	data_1m/autos_bw/auto030_rep03.png	963	0.4815
341	data_1m/autos_bw/auto020_rep05.png	888	0.4440
343	data_1m/autos_bw/auto078_rep03.png	1006	0.5030
344	data_1m/autos_bw/auto015_rep03.png	900	0.4500
351	data_1m/autos_bw/auto032_rep02.png	857	0.4285
352	data_1m/autos_bw/auto032_rep03.png	936	0.4680
355	data_1m/autos_bw/auto159_rep02.png	707	0.3535
361	data_1m/autos_bw/auto096_rep03.png	1013	0.5065
362	data_1m/autos_bw/auto096_rep04.png	1013	0.5065
363	data_1m/autos_bw/auto096_rep05.png	1013	0.5065
364	data_1m/autos_bw/auto064_rep08.png	779	0.3895
378	data_1m/autos_bw/auto169_rep01.png	536	0.2680
379	data_1m/autos_bw/auto002_rep06.png	893	0.4465
380	data_1m/autos_bw/auto130_rep02.png	981	0.4905
390	data_1m/autos_bw/auto176_rep02.png	819	0.4095
401	data_1m/autos_bw/auto026_rep03.png	773	0.3865

1.3.1 Visualización de las primeras 5 imágenes fallidas

[21]:
```
# cambiarbw_plate_rx de data_1m a data_2m
fail_df['bw_plate_rx'] = fail_df['bw_plate_rx'].apply(lambda x:
    x.replace('data_1m', 'data_2m') if isinstance(x, str) else x)
```

```
/tmp/ipykernel_10853/459876245.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  fail_df['bw_plate_rx'] = fail_df['bw_plate_rx'].apply(lambda x:
```

[22]:
```
path = 'results/2m/'

for _, row in fail_df.head(25).iterrows():
    print(f"Auto {row['num_auto']} - errores: {row['bit_errors']}")
    display(Image(filename=row['bw_plate_tx']))
    display(Image(filename=path + row['bw_plate_rx']))
```

```
Auto 128 - errores: 901
```

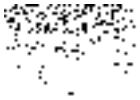
```
Auto 64 - errores: 748
```

Auto 129 - errores: 962

Auto 97 - errores: 1014

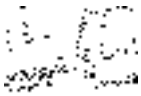
Auto 64 - errores: 793

Auto 132 - errores: 953



Auto 64 - errores: 790

Auto 66 - errores: 852



Auto 29 - errores: 616

Auto 30 - errores: 431

Auto 178 - errores: 1058

Auto 89 - errores: 1030

Auto 2 - errores: 834

Auto 36 - errores: 810

Auto 80 - errores: 724

Auto 93 - errores: 902

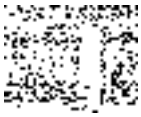
Auto 93 - errores: 902

Auto 64 - errores: 764

Auto 114 - errores: 948

Auto 126 - errores: 323

Auto 129 - errores: 972



Auto 59 - errores: 1044

Auto 104 - errores: 997

```
Auto 29 - errores: 957
```

```
Auto 63 - errores: 1019
```

[23]:
```python
# --- Umbral de bits erróneos ---
thr_err = 15

# --- Agrupar por auto y comprobar si alguna réplica cumple ---
auto_status = (
    df.groupby('num_auto')['bit_errors']
      .apply(lambda s: (s < thr_err).any())   # True = tiene al menos una buena
)

# Autos sin réplica "buena"
failed_autos = auto_status[~auto_status]      # False significa que falló
```

```
num_failed   = failed_autos.size
num_total    = auto_status.size

print(f'Autos sin ninguna réplica con < {thr_err} errores de bit: '
      f'{num_failed} de {num_total}')

# Mostrar la lista (si quieres verla)
display(failed_autos.index.tolist())
```

```
Autos sin ninguna réplica con < 15 errores de bit: 1 de 162
```

```
[169]
```